LOAD THE "RAW_SQL_TRANSACTION_RECORD.CSV" INTO POSTGRESQL DATABASE AND PERFORM THE APPROPRIATE TASK TO CLEAN AND TRANSFORM THE DATA BASED ON THE SQL DATA ANALYSIS LIFECYCLE.

```
DROP TABLE IF EXISTS transaction_records_tb;

CREATE TABLE transaction_records_tb (
  row_id INT,
  user_id NUMERIC,
  event_time TIMESTAMP,
  order_id  NUMERIC,
  product_id NUMERIC,
  category_id NUMERIC,
  category_code VARCHAR(255),
  brand VARCHAR(100),
  price DECIMAL(10,2)
);

-- loading first  data --
COPY transaction_records_tb
FROM 'C:/ProgramData/MySQL/MySQL Server
8.0/Uploads/input_files/raw_sql_transaction_record_1.csV'
DELIMITER ','
CSV HEADER;

-- confirming records loaded
SELECT * FROM transaction_records_tb LIMIT 5;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	row_id integer	user_id numeric	event_time timestamp without time zone	order_id numeric	product_id numeric	category_id numeric	category_code character varying (255)	brand character va
1	1	1520000000000000000	2020-04-24 11:50:39	2294360000000000000	1515970000000000000	2270000000000000000	electronics.tablet	samsung
2	2	1520000000000000000	2020-04-24 11:50:39	2294360000000000000	1515970000000000000	2270000000000000000	electronics.tablet	samsung
3	3	1520000000000000000	2020-04-24 14:37:43	2294440000000000000	2273950000000000000	2270000000000000000	electronics.audio.headpho...	huawei
4	4	1520000000000000000	2020-04-24 14:37:43	2294440000000000000	2273950000000000000	2270000000000000000	electronics.audio.headpho...	huawei
5	5	1520000000000000000	2020-04-24 19:16:21	2294580000000000000	2273950000000000000	2270000000000000000	[null]	karcher

```
-- loading second  data
COPY transaction_records_tb
FROM 'C:/ProgramData/MySQL/MySQL Server
8.0/Uploads/input_files/raw_sql_transaction_record_2.csv'
DELIMITER ','
CSV HEADER;

-- counting total records loaded
SELECT COUNT(*)
FROM transaction_records_tb;
```

Data Output Messages

	count bigint
1	2050142

-- checking for NULL and deleting records containing them

```
DELETE FROM transaction_records_tb
WHERE row_id IS NULL
  OR user_id IS NULL
  OR event_time IS NULL
  OR order_id  IS NULL
  OR product_id IS NULL
  OR category_id IS NULL
  OR category_code IS NULL
  OR brand IS NULL
  OR price IS NULL;
```

-- counting total records after deleting the NULL

```
SELECT COUNT(*)
FROM transaction_records_tb;
```

Data Output Messag

	count bigint
1	413996

-- removing duplicate data --

-- 1. Create a new table with unique records

```
CREATE TABLE transaction_records_temp AS
SELECT DISTINCT ON (user_id, event_time, order_id, product_id, category_id,
category_code, brand, price) *
FROM transaction_records_tb;

-- 2. Drop the old table and rename the new one
DROP TABLE transaction_records_tb;

ALTER TABLE transaction_records_temp RENAME TO transaction_records_tb;

SELECT *
FROM transaction_records_tb LIMIT 7;
```

Data Output Messages Notifications

Showing rows: 1 to 7 | Page No: 1 of 1

	row_id integer	user_id numeric	event_time timestamp without time zone	order_id numeric	product_id numeric	category_id numeric	category_code character varying (255)	brand charact
1	30495	1520000000000000000	1970-01-01 00:33:40	2340100000000000000	1515970000000000000	2270000000000000000	accessories.bag	contine
2	32586	1520000000000000000	1970-01-01 00:33:40	2340100000000000000	1515970000000000000	2270000000000000000	accessories.bag	rivacas
3	32470	1520000000000000000	1970-01-01 00:33:40	2340100000000000000	1515970000000000000	2270000000000000000	accessories.bag	samsu
4	29560	1520000000000000000	1970-01-01 00:33:40	2340100000000000000	1515970000000000000	2270000000000000000	accessories.bag	tucano
5	31586	1520000000000000000	1970-01-01 00:33:40	2340100000000000000	1515970000000000000	2270000000000000000	apparel.glove	sony
6	30882	1520000000000000000	1970-01-01 00:33:40	2340100000000000000	1515970000000000000	2270000000000000000	apparel.glove	sony
7	32421	1520000000000000000	1970-01-01 00:33:40	2340100000000000000	1515970000000000000	2270000000000000000	appliances.environment.air_hea...	ava

```
-- counting total records --
SELECT COUNT(*)
FROM transaction_records_tb;
```

-- 1 Analysing Temporal sales trend

```
SELECT
   DATE_TRUNC('Month', event_time) AS sales_date,
   COUNT(DISTINCT order_id) AS total_orders,
   ROUND(SUM(price)::numeric, 2) AS Monthly_revenue
FROM transaction_records_tb
GROUP BY 1
ORDER BY 1;
```

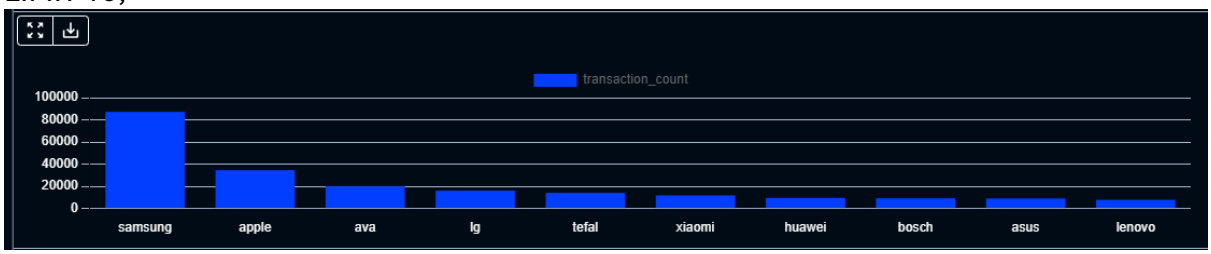
Data Output Messages Notifications

	sales_date timestamp without time zone	total_orders bigint	monthly_revenue numeric
1	1970-01-01 00:00:00	2	118706.40
2	2020-01-01 00:00:00	4	1507344.76
3	2020-02-01 00:00:00	4	1911908.37
4	2020-03-01 00:00:00	7	2514895.22
5	2020-04-01 00:00:00	118	1538162.76
6	2020-05-01 00:00:00	1873	7113987.79
7	2020-06-01 00:00:00	1646	5691553.65
8	2020-07-01 00:00:00	1988	15006857.74
9	2020-08-01 00:00:00	2145	25850112.89
10	2020-09-01 00:00:00	20	15632070.06
11	2020-10-01 00:00:00	1	17518656.02
12	2020-11-01 00:00:00	1	10686551.60

--2 Brand Analysis Top 10 brands by transaction volume

```
SELECT
```

```
  brand,
  COUNT(*) AS transaction_count,
  ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS Percentage
FROM transaction_records_tb
GROUP BY brand
ORDER BY transaction_count DESC
LIMIT 10;
```



--3 Top 10 Brands by Revenue

```
SELECT
  brand,
  COUNT(*) AS units_sold,
  ROUND(SUM(price)::numeric, 2) AS total_revenue
FROM transaction_records_tb
WHERE brand IS NOT NULL
GROUP BY brand
ORDER BY total_revenue DESC
LIMIT 10;
```

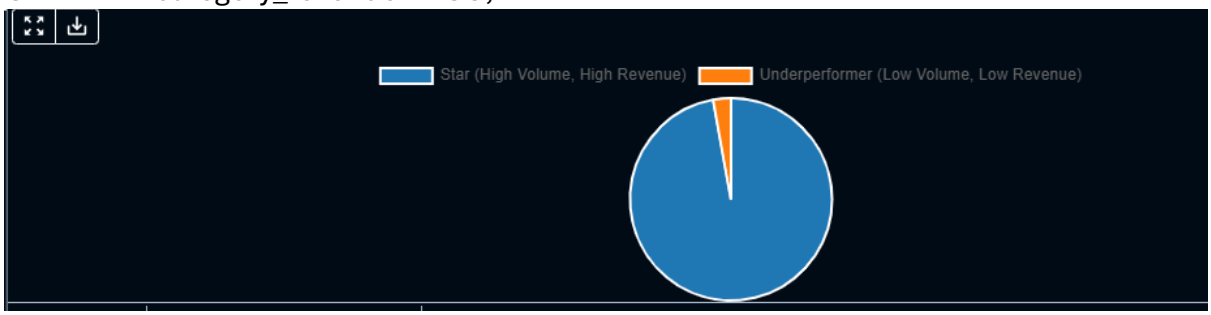
Data Output | Messages | Graph Visualiser | Notifications

	brand character varying (100)	units_sold bigint	total_revenue numeric
1	samsung	87163	28025833.22
2	apple	34553	25726217.37
3	lg	16097	7615569.33
4	asus	9046	5044060.72
5	lenovo	7832	4536824.34
6	bosch	9206	3001143.65
7	hp	6218	2452964.39
8	xiaomi	11795	2242417.14
9	huawei	9530	2111197.12
10	beko	7362	1889378.90

-- 4 Volume and Revenue by Product segmentation

```
WITH product_stats AS (
```

```
  SELECT product_id, COUNT(*) AS sale_volume, SUM(price) AS
total_revenue,AVG(price) AS avg_unit_price
  FROM transaction_records_tb
  GROUP BY product_id),averages AS (
  SELECT
    AVG(sale_volume) AS global_avg_volume,
    AVG(total_revenue) AS global_avg_revenue
  FROM product_stats
)
SELECT
  CASE
    WHEN sale_volume > global_avg_volume AND total_revenue > global_avg_revenue
THEN 'Star (High Volume, High Revenue)'
    WHEN sale_volume > global_avg_volume AND total_revenue <= global_avg_revenue
THEN 'Mass Market (High Volume, Low Revenue)'
    WHEN sale_volume <= global_avg_volume AND total_revenue > global_avg_revenue
THEN 'Premium (Low Volume, High Revenue)'
    ELSE 'Underperformer (Low Volume, Low Revenue)'
  END AS product_segment,
  COUNT(*) AS product_count,
  ROUND(SUM(total_revenue)::numeric, 2) AS category_revenue
FROM product_stats, averages
GROUP BY 1
ORDER BY category_revenue DESC;
```



SUMMARY OF SQL TRANSACTION DATA CLEANING AND ANALYSIS

Data Loading

- Two CSV files (raw_sql_transaction_record_1.csv and raw_sql_transaction_record_2.csv) were loaded into a PostgreSQL table named transaction_records_tb.
- Initial verification was performed using SELECT * LIMIT 5.

Data Cleaning

- Records containing NULL values across critical fields (row_id, user_id, event_time, order_id, product_id, category_id, category_code, brand, price) were deleted.

- Duplicate records were removed by creating a temporary table with DISTINCT values and replacing the original table.

Data Transformation and Analysis

1. Temporal Sales Trend

- Monthly aggregation of orders and revenue using DATE_TRUNC('Month', event_time).
- Metrics:
    - Total distinct orders per month.
    - Monthly revenue (rounded to two decimals).

2. Brand Analysis

- Top 10 brands ranked by transaction volume.
- Percentage contribution of each brand to total transactions.

3. Top 10 Brands by Revenue

- Ranking of brands based on total revenue.
- Metrics:
    - Units sold per brand.
    - Total revenue generated.

4. Product Segmentation (Volume and Revenue)

- Products categorized into four segments:
    - **Star**: High volume, high revenue.
    - **Mass Market**: High volume, low revenue.
    - **Premium**: Low volume, high revenue.
    - **Underperformer**: Low volume, low revenue.
- Segmentation based on comparison with global averages of volume and revenue.
- Output includes product count and category revenue per segment.

Key Outcomes

- Clean dataset free of nulls and duplicates.
- Insights into monthly sales trends.
- Identification of top-performing brands by both transaction volume and revenue.
- Strategic product segmentation for performance evaluation.